

1. Server-Side Topic: File Upload Vulnerabilities

1.1 Opis klase napada

Ranjivosti upload-a fajlova nastaju kada server ne validira pravilno fajlove koje korisnici otpremaju - njihov naziv, tip, sadržaj ili veličinu. Napadač može iskoristiti ovo da uploaduje maliciozne fajlove umesto legitimnih, najčešće serverske skripte (web shell) kojima preuzima kontrolu nad serverom.

1.2 Mogući uticaj

U najgorem slučaju napadač postavlja web shell i dobija Remote Code Execution (RCE) - može čitati/pisati fajlove, izvršavati sistemske komande i pristupati celoj infrastrukturi. Lakši scenariji uključuju Stored XSS (upload HTML/SVG fajla), prepisivanje postojećih fajlova, ili DoS napad punjenjem diska.

1.3 Ranjivosti koje su dozvolile napad

- Validacija samo Content-Type zaglavlja, koje napadač slobodno menja
- Crna lista ekstenzija umesto bele (lako se zaobiđe alternativnim ekstenzijama)
- Server izvršava kod u upload direktorijumu
- Dozvoljeno uploadovanje .htaccess / web.config fajlova
- Naziv fajla se ne sanitizuje (path traversal, null bajtovi)

1.4 Kontramere

- Koristiti belu listu dozvoljenih ekstenzija (npr. samo .jpg, .png, .pdf)
- Proveravati stvarni sadržaj fajla (magični bajtovi), ne samo zaglavlje
- Preimenovati svaki fajl pri čuvanju u random ime
- Konfigurisati server da ne izvršava kod u upload direktorijumu
- Čuvati fajlove van web root-a i servirati ih kontrolisano
- Ograničiti veličinu fajla

2. Writeup-ovi zadataka - File Upload Vulnerabilities

Lab 1: Remote code execution via web shell upload (*Apprentice*)

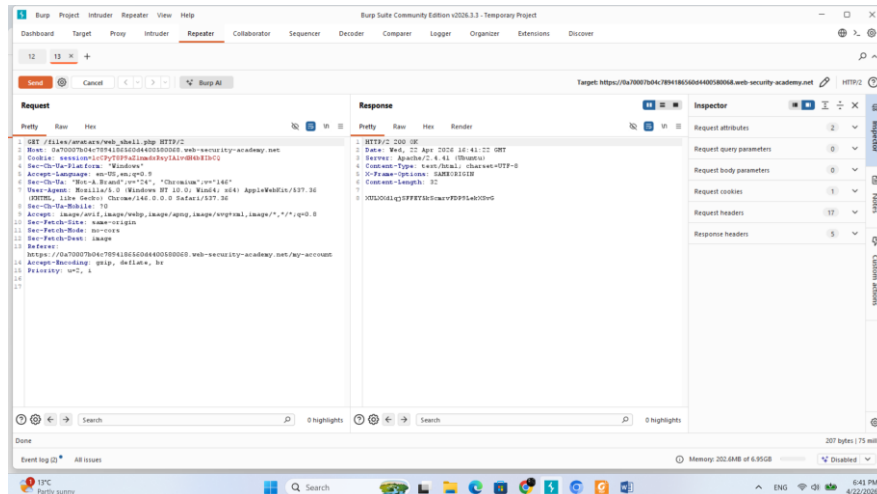
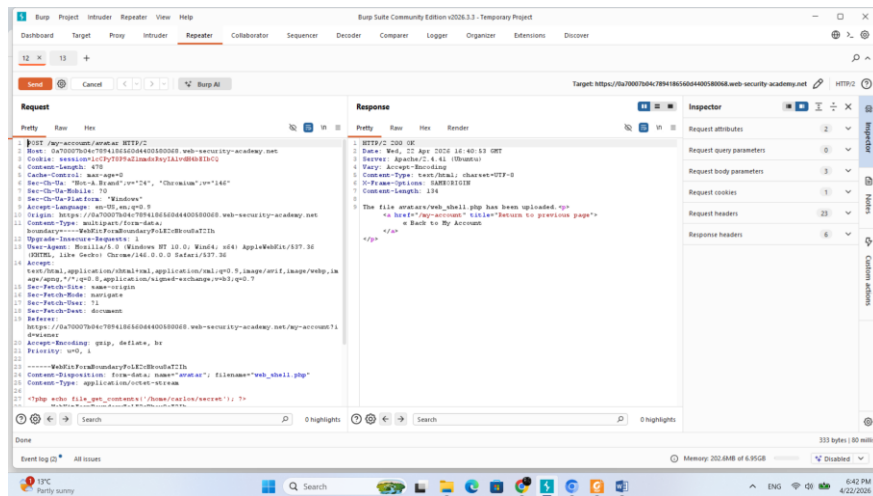
Cilj: Uploadovati PHP web shell na server koji nema nikakvu validaciju fajlova i pročitati sadržaj /home/carlos/secret.

Rešenje:

1. Prijava na nalog (wiener:peter) i odlazak na stranicu naloga gde postoji opcija za upload avatara.
2. Upload legitimne slike kako bi se videlo kako funkcioniše zahtev - u Burp Proxy istoriji pronađen GET /files/avatars/<ime-slike> zahtev, poslat u Repeater.
3. Kreiran fajl web_shell.php sa sledećim sadržajem:

```
<?php echo file_get_contents('/home/carlos/secret'); ?>
```

Ovaj fajl uploadovan kao avatar - server ga prihvata bez ikakve validacije.



5. U Repeater-u, putanja promenjena u GET /files/avatars/web_shell.php - server izvršava skriptu i vraća sadržaj tajnog fajla u response-u.

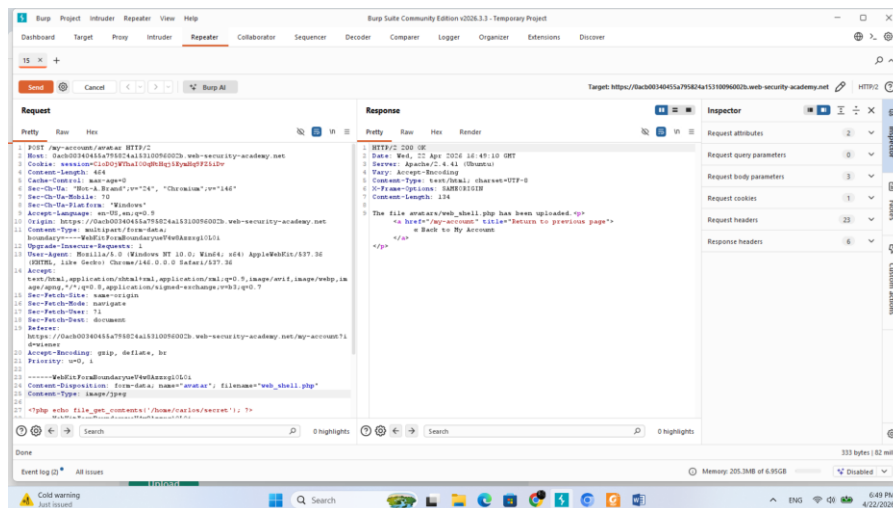
Rezultat: Server je direktno izvršio PHP skriptu i vratio sadržaj fajla /home/carlos/secret, čime je lab rešen.

Lab 2: Web shell upload via Content-Type restriction bypass (*Apprentice*)

Cilj: Server proverava Content-Type zaglavlje i dozvoljava samo image/jpeg i image/png - zaobići ovu proveru i uploadovati PHP web shell.

Rešenje:

1. Pokušan direktan upload web_shell.php - server odbija jer Content-Type nije slika.
2. Upload zahtev (POST /my-account/avatar) pronađen u Burp istoriji i poslat u Repeater.
3. U Repeater-u, u delu zahteva koji se odnosi na fajl, Content-Type promenjen iz application/x-php u image/jpeg - naziv fajla ostaje web_shell.php.



4. Zahtev poslat - server prihvata fajl jer veruje deklarisanom zaglavlju bez provere stvarnog sadržaja.
5. U drugom Repeater tabu, GET zahtev upućen na /files/avatars/ web_shell.php - vraćen sadržaj tajnog fajla.

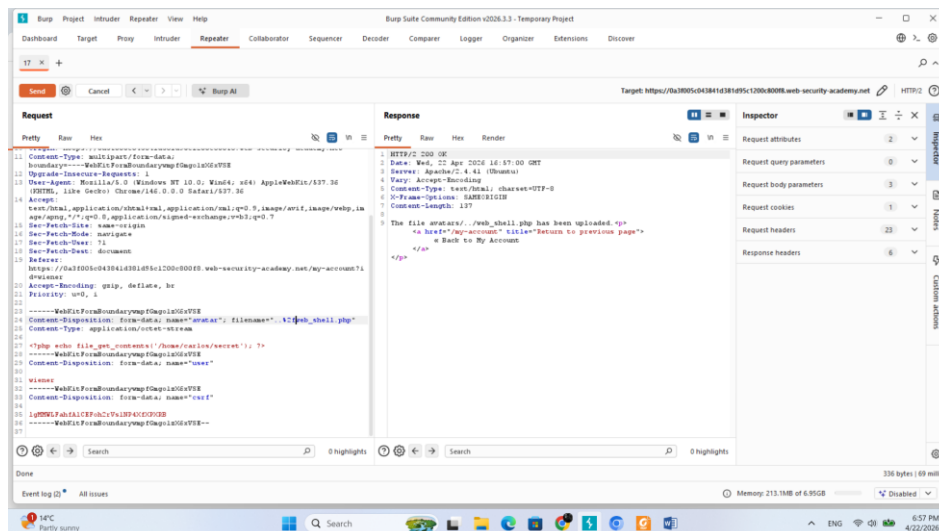
Rezultat: Prosta izmena HTTP zaglavlja u Burp-u zaobišla je validaciju, jer server nije proveravao stvarni sadržaj fajla.

Lab 3: Web shell upload via path traversal (*Practitioner*)

Cilj: Server ne dozvoljava izvršavanje koda u upload direktorijumu - pronaći način da se fajl smesti u drugi direktorijum gde izvršavanje nije blokirano.

Rešenje:

1. Upload web_shell.php prolazi, ali GET zahtev na /files/avatars/web_shell.php vraća sadržaj fajla kao plain text umesto da ga izvršava - server blokira egzekuciju u tom direktorijumu.
2. U upload POST zahtevu pronađenom u Burp istoriji, u Content-Disposition zaglavlju promenjen filename u ../web_shell.php - pokušaj path traversal-a.
3. Server odgovara sa The file avatars/web_shell.php has been uploaded - znači da je obrisao ../ sekvencu.
4. Isti pokušaj sa URL enkodiranom kosom crtom: filename="..%2fweb_shell.php".



5. Server sada odgovara sa The file avatars/../../web_shell.php has been uploaded - traversal sekvenca je prošla.
6. GET zahtev na /files/web_shell.php (jedan nivo iznad avatars/) vraća izvršen rezultat skripte.

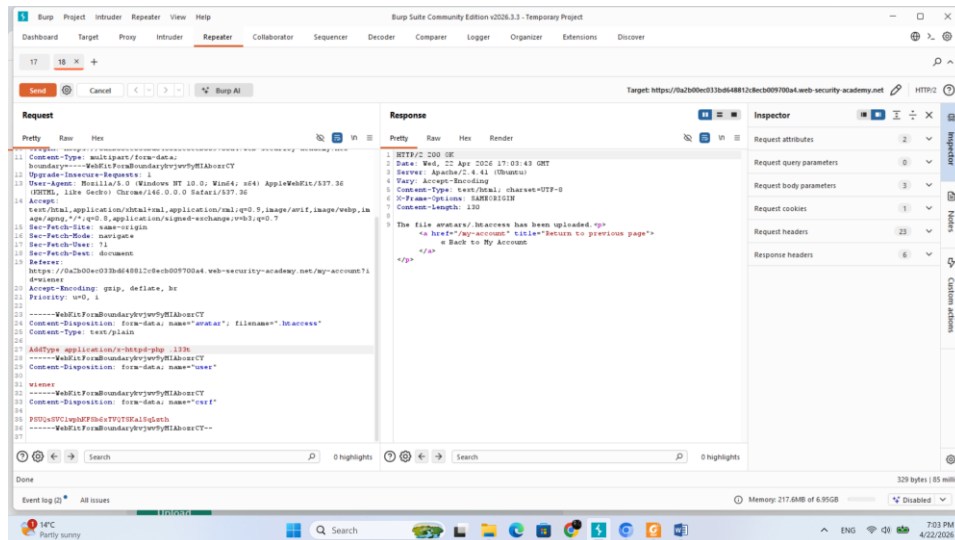
Rezultat: URL enkodiranje kose crte zaobišlo je filtriranje naziva fajla i omogućilo postavljanje web shell-a van zaštićenog direktorijuma.

Lab 4: Web shell upload via extension blacklist bypass (*Practitioner*)

Cilj: Server blokira .php ekstenziju - zaobići crnu listu uploadovanjem .htaccess fajla koji mapira custom ekstenziju na PHP izvršavanje.

Rešenje:

1. Direktan upload web_shell.php odbijen - server javlja da .php nije dozvoljena ekstenzija.
2. Iz response zaglavlja utvrđeno da se radi o Apache serveru.
3. U Burp Repeater-u, upload zahtev modifikovan da uploaduje .htaccess fajl umesto slike:
 - filename → .htaccess
 - Content-Type → text/plain
 - Sadržaj fajla → AddType application/x-httpd-php .l33t



4. .htaccess uspešno uploadovan - Apache sada tretira .l33t fajlove kao PHP.
5. Isti upload zahtev ponovo modifikovan: filename → web_shell.l33t, sadržaj → originalna PHP skripta.
6. GET zahtev na /files/avatars/ web_shell.l33t - server izvršava fajl i vraća tajni sadržaj.

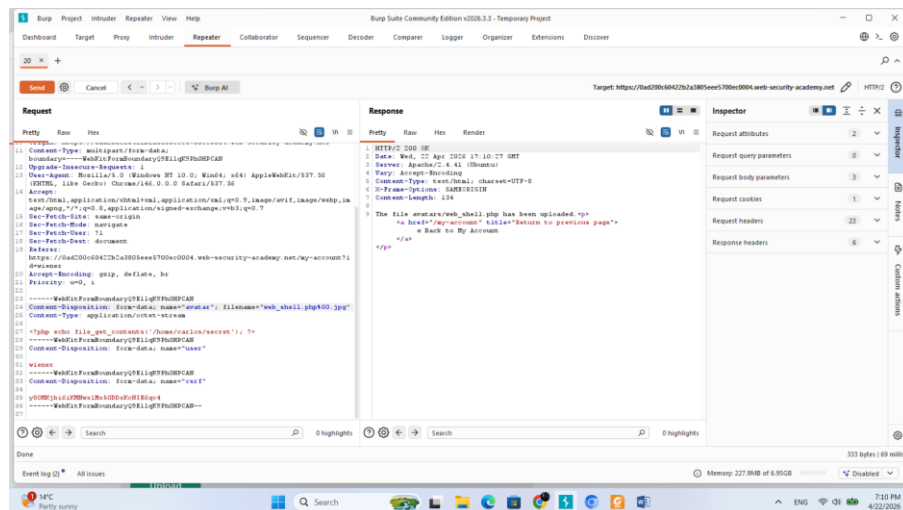
Rezultat: Uploadovanjem malicioznog .htaccess fajla redefinisano je ponašanje Apache servera za custom ekstenziju, čime je crna lista u potpunosti zaobiđena.

Lab 5: Web shell upload via obfuscated file extension (*Practitioner*)

Cilj: Server dozvoljava samo .jpg i .png fajlove - zaobići proveru obfuskacijom ekstenzije null bajtom.

Rešenje:

1. Direktan upload web_shell.php odbijen - dozvoljena su samo JPG i PNG.
2. Upload POST zahtev pronađen u Burp istoriji i poslat u Repeater.
3. U Content-Disposition zaglavlju, filename promenjen u web_shell.php%00.jpg - null bajt između PHP ekstenzije i .jpg sufiksa.



4. Server prihvata fajl - validacija vidi .jpg na kraju, ali pri čuvanju fajla null bajt terminira string pa se fajl čuva kao web_shell.php.
5. GET zahtev na /files/avatars/web_shell.php vraća izvršen sadržaj tajnog fajla.

Rezultat: Null bajt obfuskacija iskoristila je razliku u obradi stringa između validacionog koda (high-level jezik) i sistema za čuvanje fajla (low-level C funkcije), koje tretiraju null bajt kao kraj stringa.

3. Client-Side Topic: Cross-Site Scripting (XSS)

3.1 Opis klase napada

Cross-site scripting (XSS) je ranjivost koja napadaču omogućava da ubaci i izvrši maliciozni JavaScript kod u browseru druge osobe - žrtve. Napad funkcioniše tako što se manipuliše ranjivom aplikacijom da vrati maliciozni skript korisniku, koji se zatim izvršava u kontekstu njegove sesije, zaobilazeći same-origin policy.

Postoje tri vrste XSS napada:

- **Reflected XSS** - maliciozni skript dolazi iz trenutnog HTTP zahteva. Napadač konstruiše maliciozni URL (npr. sa <script> tagom u query parametru), žrtva klikne na link, server reflektuje taj input nazad u response i browser ga izvršava.
- **Stored XSS** - maliciozni skript se čuva u bazi podataka aplikacije (npr. kao komentar na blogu) i izvršava se svaki put kada bilo koji korisnik poseti tu stranicu. Opasniji je jer ne zahteva da žrtva klikne na poseban link.
- **DOM-based XSS** - ranjivost postoji isključivo u client-side JavaScript kodu. Aplikacija čita podatke iz nepouzdanog izvora (npr. location.search, document.cookie) i upisuje ih u DOM na nesiguran način, bez ikakvog učešća servera.

3.2 Mogući uticaj

Uticaj XSS napada zavisi od privilegija kompromitovanog korisnika i osetljivosti aplikacije:

- **Krađa sesijskih kolačića** - napadač može ukrasti session cookie i preuzeti nalog žrtve
- **Keylogging i krađa kredencijala** - maliciozni skript može snimati tastere i krasti lozinke
- **Izvođenje akcija u ime žrtve** - sve što korisnik može da uradi u aplikaciji, može i napadač (promena lozinke, slanje poruka, finansijske transakcije)
- **Defacement** - izmena izgleda stranice
- **Phishing** - ubacivanje lažnih formi za prijavu unutar legitimne stranice
- **Širenje napada** - ako kompromitovani korisnik ima administratorske privilegije, napadač može potpuno preuzeti kontrolu nad aplikacijom i svim njenim korisnicima

3.3 Ranjivosti koje su dozvolile napad

- Korisnički unos se direktno uključuje u HTML response bez enkodiranja (reflected/stored XSS)

- Aplikacija čuva nekontrolisani korisnički unos u bazi i prikazuje ga drugima bez sanitizacije (stored XSS)
- Client-side JavaScript čita podatke iz nepouzdanih izvora (location.hash, location.search, document.referrer) i prosleđuje ih opasnim sinkovima poput innerHTML, document.write(), eval() (DOM XSS)
- Odsustvo ili pogrešna konfiguracija Content Security Policy (CSP)
- Upotreba crne liste zabranjenih karaktera/tagova umesto validacije bele liste

3.4 Kontramere

- **Enkodirati output** - sve korisničke podatke koji se prikazuju u HTML-u enkodovati prema kontekstu: HTML enkodiranje za HTML sadržaj, JavaScript enkodiranje za JS kontekst, URL enkodiranje za URL-ove. Ovo je najvažnija mera.
- **Validirati input** - na serveru prihvatati samo očekivane vrednosti (bela lista), odbacivati ili čistiti sve ostalo
- **Koristiti sigurne DOM API-je** - umesto innerHTML i document.write() koristiti textContent i createElement() koji ne parsuju HTML
- **Content Security Policy (CSP)** - HTTP zaglavlje koje browseru govori odakle sme da učitava i izvršava skripte; kao poslednja linija odbrane značajno smanjuje uticaj čak i ako XSS postoji
- **HttpOnly flag na kolačićima** - sprečava JavaScript da pristupi session cookie-ju, čime se eliminiše najčešći cilj XSS napada
- **Koristiti moderne template engine-e** koji po defaultu enkoduju output (npr. React, Angular)
- **Zaglavlje X-Content-Type-Options: nosniff** - sprečava browser da pogađa tip sadržaja

4. Writeup-ovi zadataka - Cross-Site Scripting (XSS)

Lab 1: Reflected XSS into HTML context with nothing encoded (*Apprentice*)

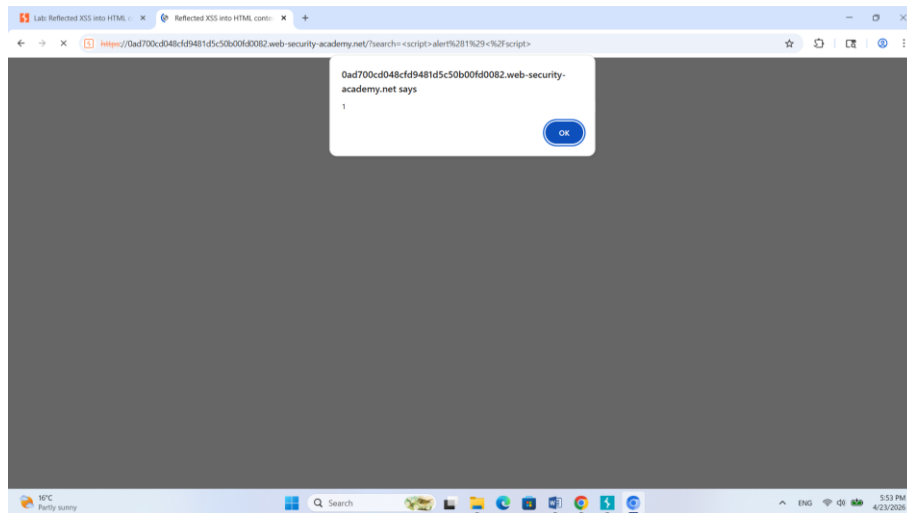
Cilj: Iskoristiti reflected XSS ranjivost u search funkcionalnosti i izvršiti `alert()`.

Rešenje:

1. U search box unet sledeći payload:

```
<script>alert(1)</script>
```

2. Kliknuto "Search" - server reflektuje unos direktno u HTML response bez ikakvog enkodiranja.



Rezultat: Browser je izvršio ubačeni skript jer server nije enkodovao korisnički unos pre uključivanja u HTML stranicu.

Lab 2: Stored XSS into HTML context with nothing encoded (*Apprentice*)

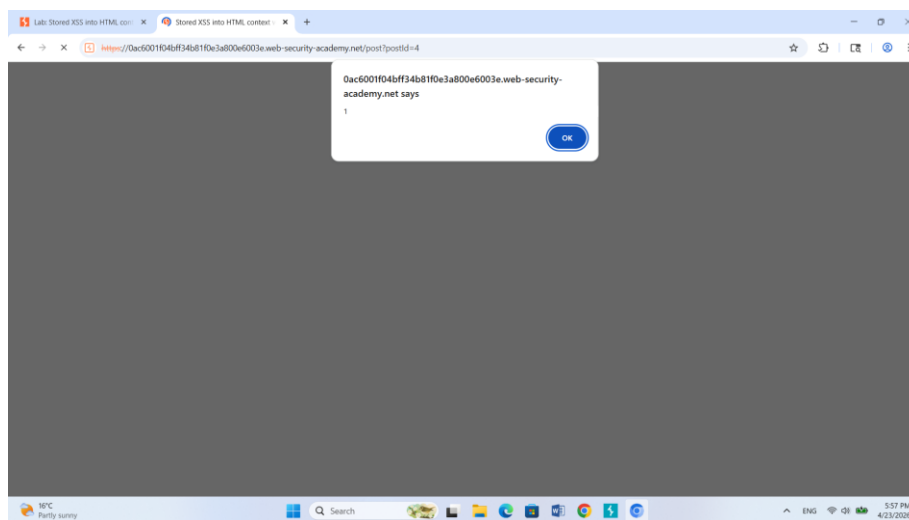
Cilj: Ubaciti XSS payload u komentar na blogu koji će se izvršiti kada drugi korisnik poseti stranicu.

Rešenje:

1. Otvorena stranica blog posta sa sekcijom za komentare.
2. U polje za komentar unet payload:

```
<script>alert(1)</script>
```

3. Popunjena obavezna polja (ime, email, website) i kliknut "Post Comment".
4. Povratak na blog stranicu - pri učitavanju stranice browser izvršava sačuvani skript.



Rezultat: Payload je sačuvan u bazi i izvršava se kod svakog korisnika koji poseti stranicu, za razliku od reflected XSS-a koji zahteva klik na poseban link.

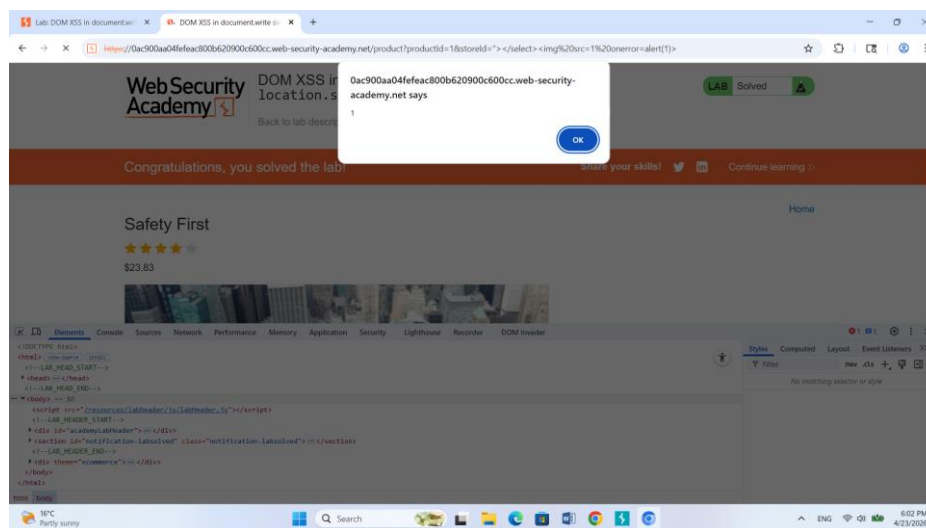
Lab 3: DOM XSS in document.write sink using source location.search inside a select element *(Practitioner)*

Cilj: Iskoristiti DOM XSS ranjivost gde JavaScript čita storeId parametar iz URL-a i upisuje ga u DOM putem document.write, unutar <select> elementa.

Rešenje:

1. Na stranici proizvoda primećeno da JavaScript čita storeId parametar iz location.search i koristi document.write da ga ubaci u <select> dropdown za proveru zaliha.
2. URL modifikovan dodavanjem test vrednosti (?storeId=test) - potvrđeno u DevTools-u da se vrednost pojavljuje unutar <select> taga.
3. URL promenjen sa XSS payloadom koji izlazi iz <select> elementa:

product?productId=1&storeId=""></select>



Rezultat: Payload je zatvorio `</select>` tag i ubacio `<img>` tag čiji onerror handler poziva `alert()` - sve bez ikakve interakcije sa serverom.

Lab 4: Reflected DOM XSS (Practitioner)

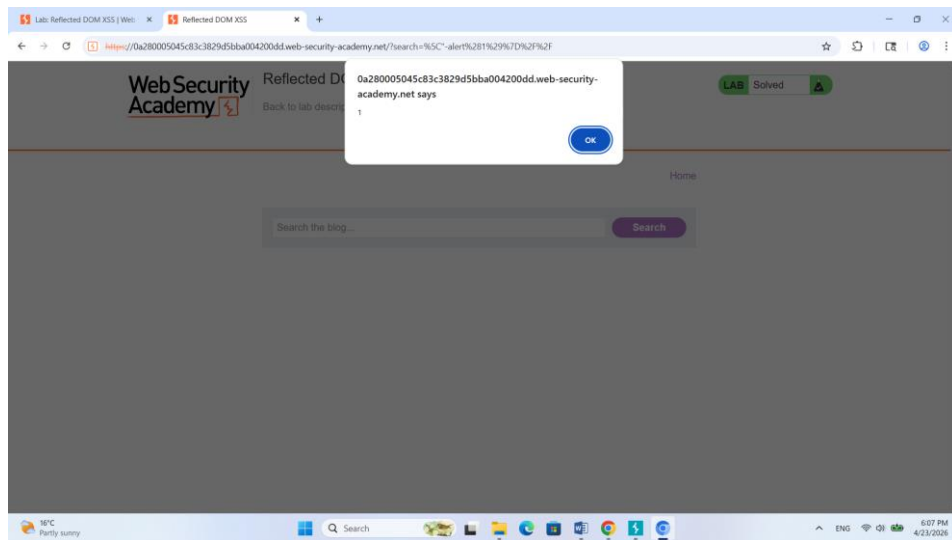
Cilj: Server reflektuje korisnički unos u JSON response koji se zatim obrađuje eval() funkcijom na klijentskoj strani - konstruisati payload koji izlazi iz JSON stringa i izvršava alert().

Rešenje:

1. Search funkcionalnost korišćena sa test stringom XSS - u Burp-u primećeno da server vraća JSON response (search-results) koji sadrži uneseni pojam.
2. Analizom searchResults.js fajla utvrđeno da se JSON response prosleđuje eval() funkciji.
3. Testiranjem utvrđeno da server enkoduje navodnik ("), ali ne enkoduje backslash (\).
4. Payload koji iskorišćava ovu nedoslednost:

```
\"-alert(1)}//
```

Ubačeni backslash neutralizuje server-side escape navodnika, što rezultuje dvostrukim backslashom (\\) koji poništava escaping. Navodnik ostaje neeskejpovan i zatvara JSON string, - razdvaja izraze, alert(1) se izvršava, a }// zatvara JSON objekat i komentariše ostatak.



Rezultat: Kombinacija server-side refleksije i nesigurne eval() upotrebe na klijentskoj strani dozvolila je izlazak iz JSON konteksta i izvršavanje proizvoljnog JavaScript koda.

Lab 5: Stored DOM XSS (*Practitioner*)

Cilj: Iskoristiti stored DOM XSS u sekciji za komentare gde JavaScript filter zamenjuje ugaone zagrade, ali samo prvu pojavu.

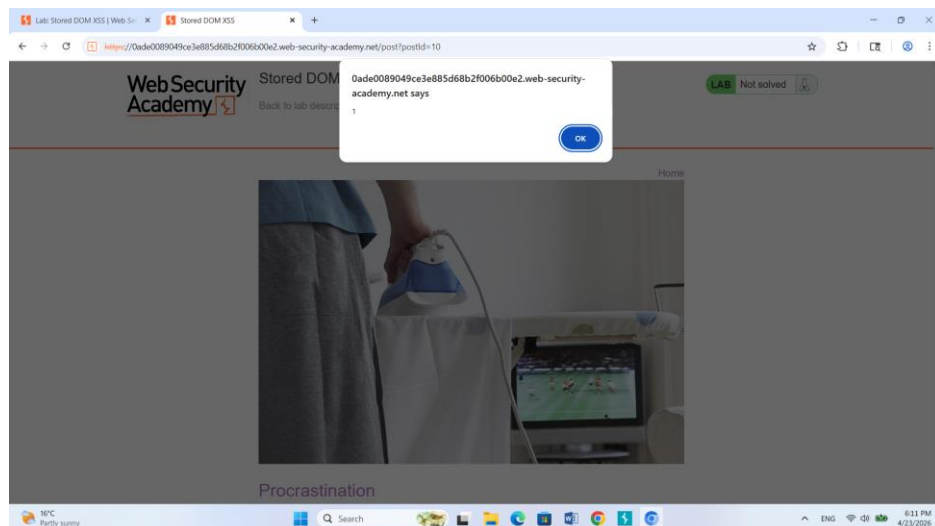
Rešenje:

1. Analizom JavaScript koda utvrđeno da aplikacija koristi replace() funkciju za enkodiranje < i > karaktera kao zaštitu od XSS-a.
2. Problem: kada se replace() poziva sa stringom kao prvim argumentom (umesto regex-a sa /g flagom), zamenjuje **samo prvu pojavu** karaktera.
3. Payload koji iskorišćava ovu grešku:

```
<><img src=1 onerror=alert(1)>
```

Prve <> zagrade bivaju enkodovane (filter "potroši" svoju zamenu na njih), ali sve naredne zagrade ostaju netaknute i browser ih parsuje kao validan HTML tag.

4. Komentar sačuvan - pri ponovnoj poseti stranice onerror handler izvršava alert().



Rezultat: Nepotpuna regex implementacija (replace() bez /g flaga) dozvolila je zaobilaženje filtera prostim dodavanjem "žrtvenih" zagrada na početak payloada.